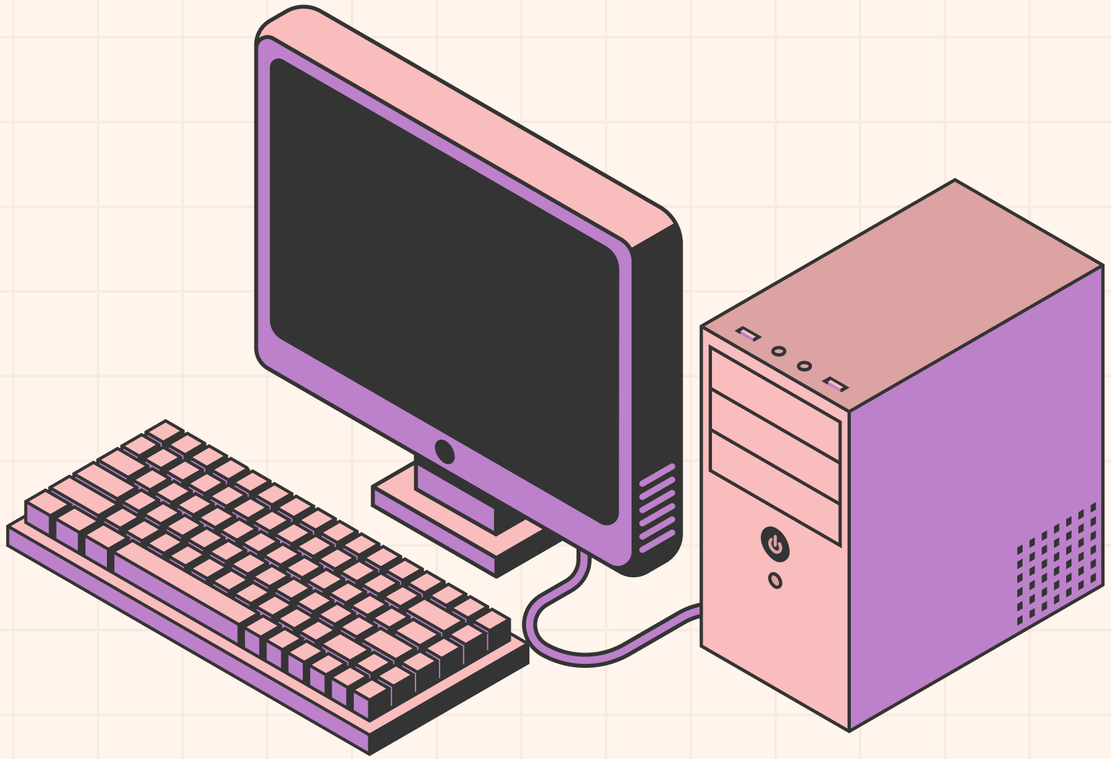
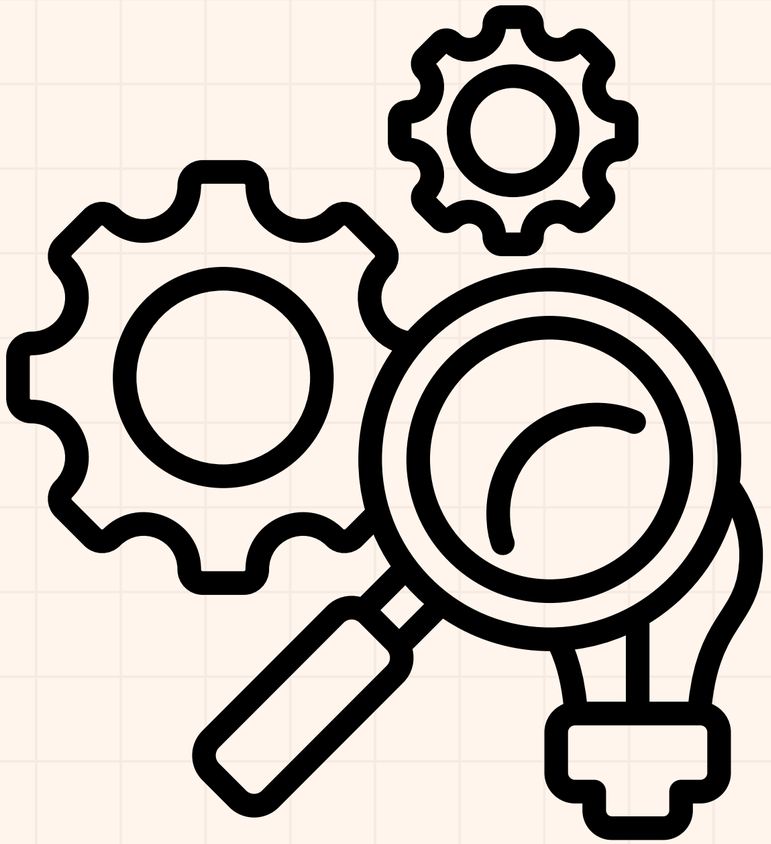
MODELOS DE DESARROLLO Y DEVSECOPS

Criptografía y Seguridad



¿EXISTE UNA METODOLOGÍA SUPERIOR?

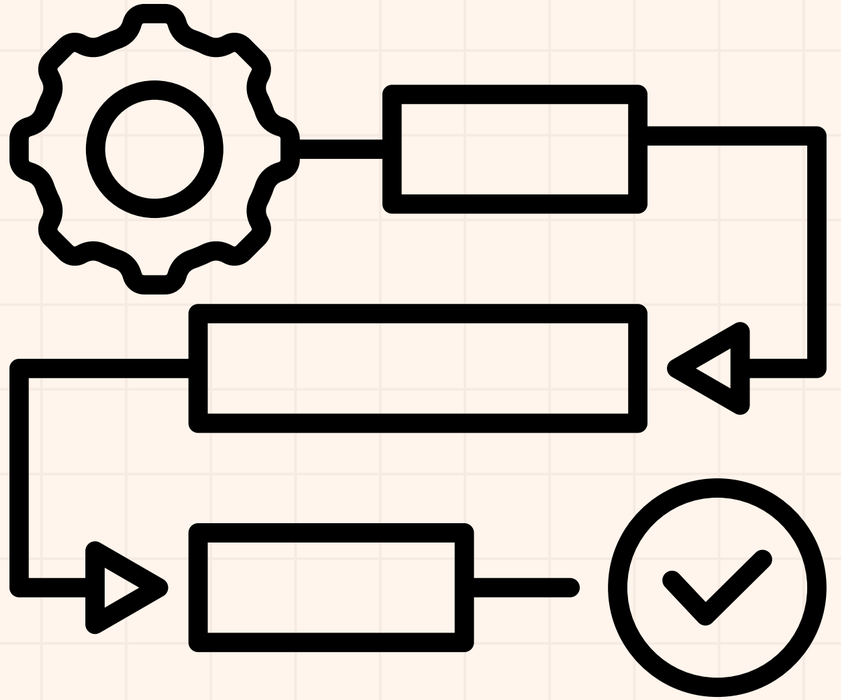
Ningún modelo es universalmente perfecto. La elección depende del nivel de incertidumbre, estabilidad de requisitos, criticidad del sistema, normatividad y riesgos del entorno.



MODELO EN CASCADA

El proyecto se divide en distintas fases secuenciales, donde el equipo puede pasar a la siguiente fase sólo cuando se haya completado la anterior. El Ideal para requisitos estables, entornos altamente regulados y contratos rígidos.

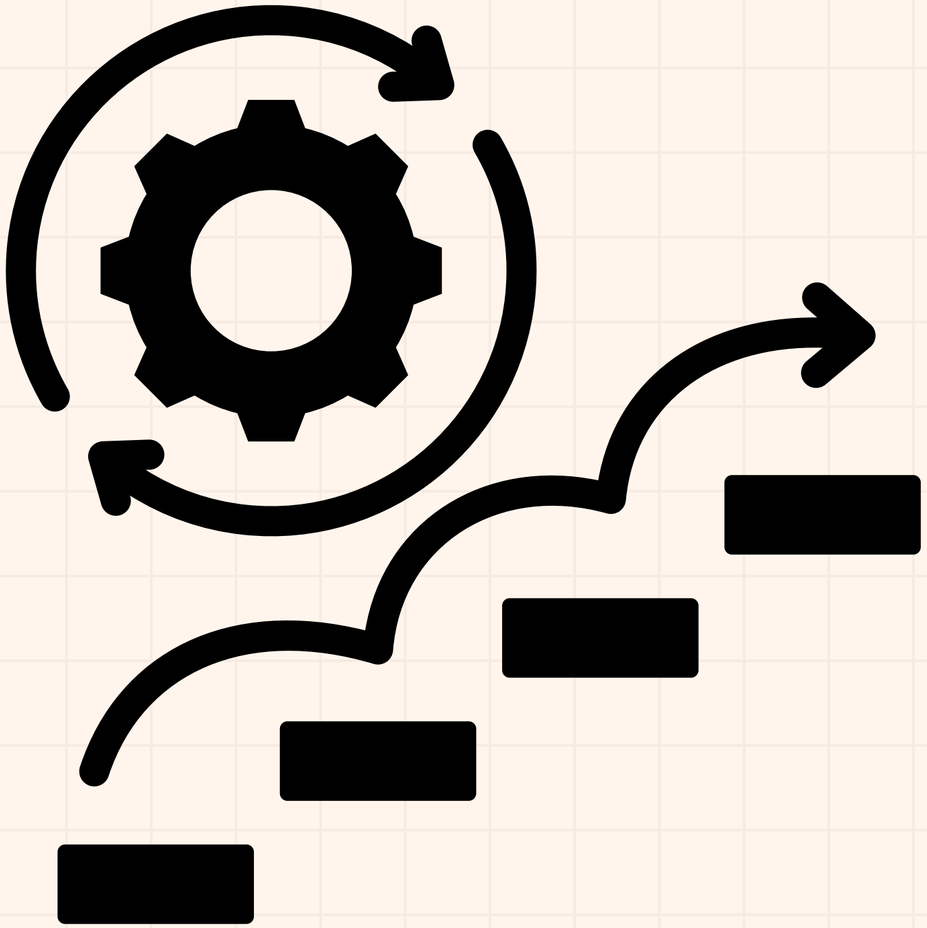
Descubrir fallas de arquitectura o vulnerabilidades críticas al final genera retrasos y sobrecostos masivos.



MODELO INCREMENTAL

Este modelo descompone un proyecto en una sucesión de agregados denominados incrementos. Estos agregados conforman un fragmento de la funcionalidad total del producto. Entrega valor temprano y permite validar módulos en producción.

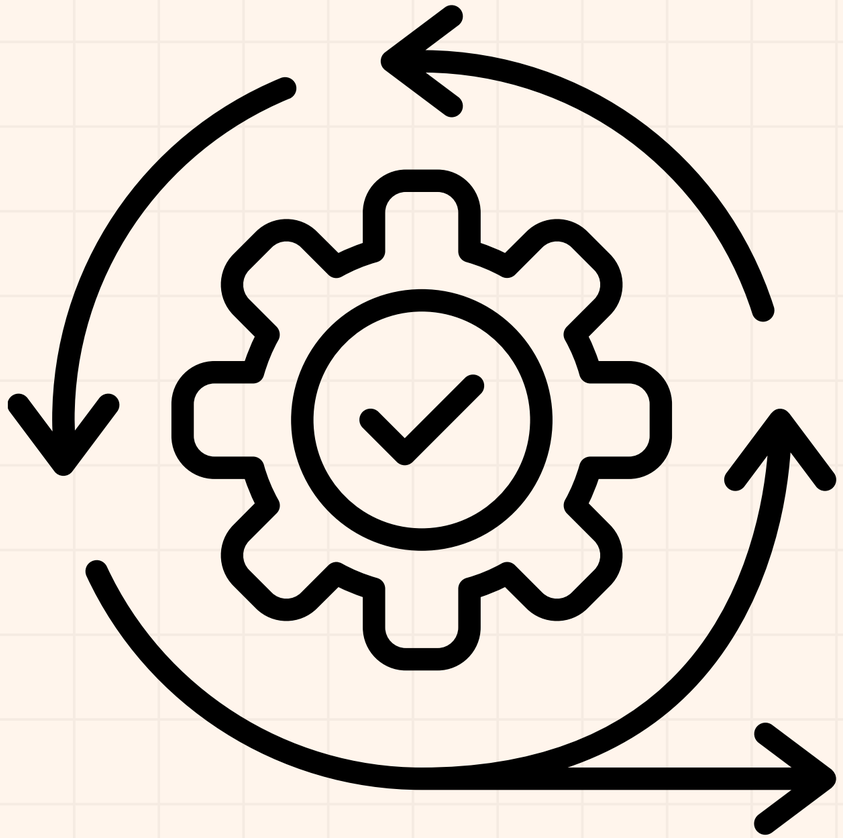
Requiere una arquitectura inicial sólida para evitar reescribir esquemas de acceso e integración.



MODELO ITERATIVO

No se agrega solo piezas nuevas; se revisa y mejora continuamente lo ya existente.. Cada iteracion es un mini-proyecto autocontenido que resulta en una version funcional del software.

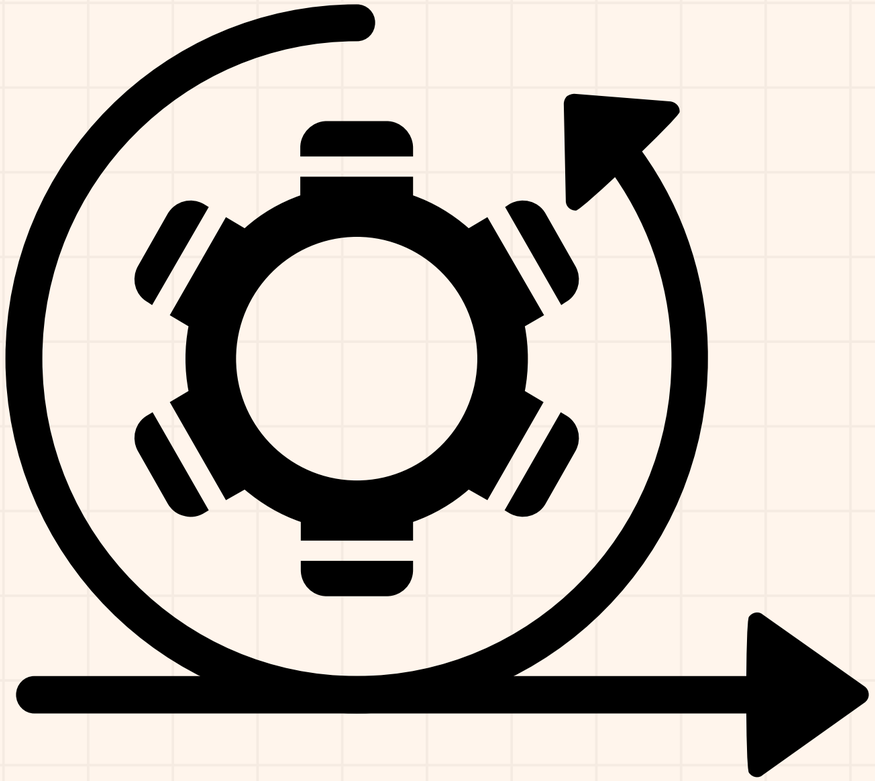
Permite endurecer controles y parchear debilidades en cada ciclo sucesivo.



MODELO EN ESPIRAL

El desarrollo esta basado en el riesgo, Cada ciclo evalúa alternativas y mitiga incertidumbres técnicas antes de construir.

Fundamental en arquitecturas médicas, aeroespaciales y sistemas con criptografía experimental.



ACTIVIDAD

¿Qué enfoque describe mejor las siguientes oraciones?

- **“Entregamos primero inscripción y después expedientes”.**
- **“Cada dos semanas mejoramos el mismo módulo”.**
- **“Primero analizamos los riesgos técnicos más importantes y construimos prototipos”.**
- **“Terminamos requisitos antes de comenzar diseño”.**

PROTOTIPADO

Antes de iniciar a desarrollar, se nesesita saber si la idea que se tiene cumple con lo que se nesesita, para esto se realiza un prototipo. Un prototipo rápido valida necesidades de usuarios y tecnología, pero nunca debe pasar directamente a producción sin arquitectura ni seguridad.

Este prototipo se construye con la idea de ser desechado posteriormente, sin embargo; el prototipo puede ser evolucionado hasta convertirse en una parte del producto

METODOLOGIAS ÁGILES

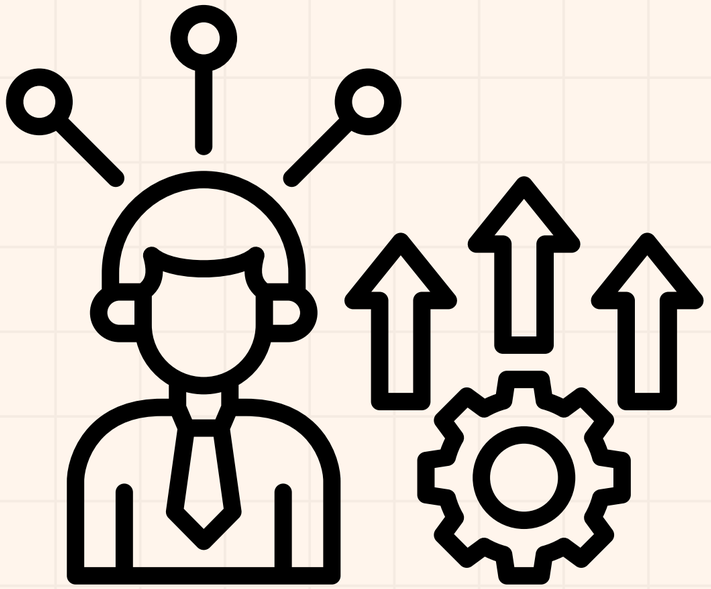
Permiten adaptar la forma de trabajo a las condiciones del proyecto, consiguiendo flexibilidad e inmediatez en la respuesta para amoldar el proyecto y su desarrollo a las circunstancias específicas del entorno.

En este contexto “ágil” no significa trabajar sin planeación o sin diseño en mente, implica una mayor supervisión, incrementos frecuentes y trabajo priorizado.

SCRUM: CICLOS CORTOS Y ADAPTACIÓN

marco de trabajo ágil diseñado para ayudar a equipos a trabajar en proyectos complejos, entregando productos funcionales de manera iterativa y continua.

Se estructura en torno a ciclos cortos de trabajo llamados sprints, que generalmente duran entre una y cuatro semanas. se basa en varios principios fundamentales que guían su implementación:



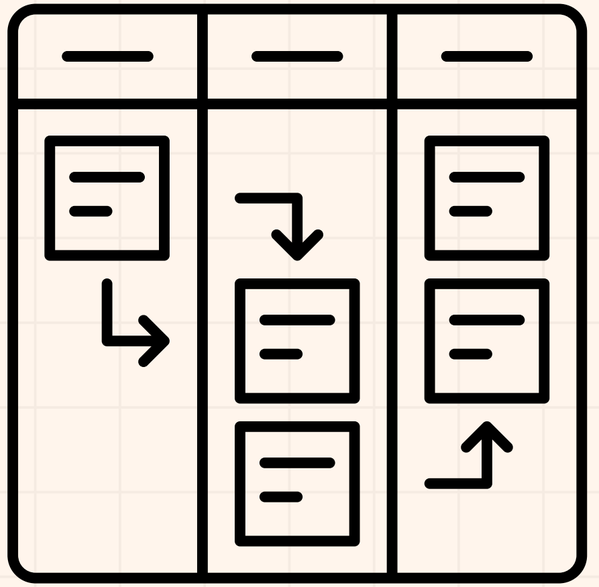
- Transparencia
- Inspección
- Adaptación
- Iteraciones rápidas

KANBAN

Kanban permite ajustar el trabajo en progreso o **work in progress (WIP)** a la capacidad del equipo, a través de las etapas de desarrollo que se definan.

Limita el WIP para evitar cuellos de botella y reducir cambios de contexto en equipos de soporte y SOC.

La metodología se implementa por medio de *tableros Kanban*: muestran el trabajo organizado por columnas. Cada columna representa una etapa, por ejemplo, Trabajo pendiente, En progreso y Terminado, y las tareas avanzan de una a otra hasta completarse.



PROGRAMACIÓN EXTREMA (XP)

Busca mejorar la calidad del producto y la capacidad de adaptación a los cambios, se caracteriza por una serie de prácticas que fomentan la colaboración y la eficiencia del equipo de desarrollo. Se caracteriza por:

- Ciclos de retroalimentación cortos
- Desarrollo guiado por pruebas (TDD)
- Programación en pareja
- Refactorización continua
- Involucramiento del cliente

DEVOPS

Conjunto de prácticas y cultura orientadas a mejorar la colaboración entre desarrollo y operaciones, utilizando automatización y retroalimentación frecuente para entregar software de manera confiable.

- Reducir fricción.
- Automatizar.
- Integrar.
- Entregar con frecuencia.
- Detectar problemas temprano.
- Mejorar confiabilidad.

DE DEVOPS A DEVSECOPS

En el modelo tradicional, la seguridad era un policía que bloqueaba los despliegues al final. DevOps integró Desarrollo y Operaciones; DevSecOps incorpora Seguridad como socio automatizado y transversal. El principio central es: “Seguridad desde el inicio y durante todo el proceso”, para lograr esto se utiliza Shift left (priorizar la seguridad primero) y Shift right (continuar evaluando la seguridad después del despliegue).

EL PIPELINE DEVSECOPS:

1.- **CÓDIGO**
MODELADO DE AMENAZAS

2.- **SAST & SCA**
ESCANEO DE CÓDIGO ESTÁTICO Y ANÁLISIS DE LIBRERÍAS ABIERTAS.

3.- **BUILD**
HARDENING DE CONTENEDORES Y FIRMA DIGITAL DE IMÁGENES

4.- **DAST**
PRUEBAS DINÁMICAS CONTRA APIS Y APPS EN EJECUCIÓN

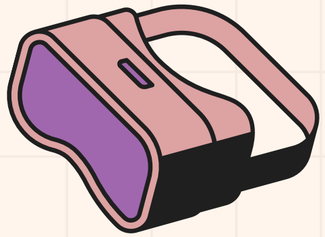
5.- **OPERACIÓN**
SHIFT RIGHT: MONITOREO CON SIEM/SOC Y TELEMETRÍA ACTIVA.

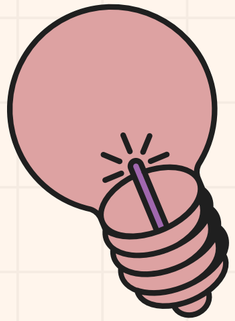
Contexto del Proyecto	Incertidumbre	Regulación / Riesgo	Enfoque Recomendado
Migración de Infraestructura / Data Center	Baja (Requisitos fijos)	Alta normatividad	Cascada
Nueva Plataforma Web Institucional	Media / Alta	Datos personales (Media)	Scrum / Incremental
Innovación en Algoritmos / Criptografía	Muy Alta (Técnica)	Riesgo técnico crítico	Espiral / Prototipado
Atención de Incidentes	Flujo continuo	Operación crítica diaria	Kanban
Banca Digital y Transaccional	Media (Entregas continuas)	Muy Alta (PCI-DSS)	Cascada + DevSecOps + CI/CD

SEGURIDAD: TRADICIONAL VS. ÁGIL

Los enfoques tradicionales se basan en “Controles por puertas”, puntos de aprobación formal antes de pasar de fase, además requieren documentación rigurosa, es decir; especificaciones formales de control, actas de entrega y auditorías de cierre.

Por otro lado; en los enfoques modernos se basan en la seguridad como flujo continuo, es decir criterios de seguridad embebidos en el Backlog, pruebas automáticas y DoD, esto con el proposito de tener un enfoque “Fallar Temprano” (Fail Fast) que basicamente nos dice que encontrar una falla previa al despliegue es mucho mejor que encontrarla en produccion.





GRACIAS

